

Q1) Range Sum Queries

Problem:

You are given an array of N integers. You are asked Q queries. Each query provides two indices L and R . For each query, efficiently compute:

$$\sum_{i=L}^R A[i]$$

Idea:

Use prefix sums:

$$P[i] = A[1] + A[2] + \dots + A[i]$$

Then each query can be answered in $O(1)$:

$$\text{Sum}(L, R) = P[R] - P[L - 1]$$

Q2) Longest Subarray with Sum $\leq X$

Problem:

Given an array of positive integers and a target value X , find the length of the longest contiguous subarray whose sum does not exceed X .

Idea:

Use two pointers (sliding window). Expand right pointer and shrink left pointer when sum exceeds X . Maintain maximum valid window length.

Q3) Factory Machines

Problem:

You have N machines. The i -th machine takes t_i time to produce one product. All machines work in parallel. Find the minimum time required to produce exactly T products.

Idea:

Use binary search on time T_{time} . For a fixed time t , total products produced is:

$$\sum_{i=1}^N \left\lfloor \frac{t}{t_i} \right\rfloor$$

Check feasibility and minimize using binary search.

Q4) Bit Manipulation: Sum of Pairwise XOR

Theory:

Numbers are represented in binary. Bitwise operations:

- AND (&): 1 if both bits are 1
- OR (|): 1 if at least one bit is 1
- XOR ($\oplus$): 1 if bits differ
- Left shift (<<): multiply by 2^k
- Right shift (>>): divide by 2^k

Problem:

Given an array of N integers, compute:

$$\sum_{1 \leq i < j \leq N} (A[i] \oplus A[j])$$

Idea:

For each bit position, count number of 1s and 0s. Contribution:

$$\text{bit contribution} = (\text{ones}) \cdot (\text{zeros}) \cdot 2^b$$

Sum over all bit positions gives $O(N \log A)$ solution.

Q5) Greedy: Interval Scheduling

Theory:

A greedy algorithm makes the locally optimal choice at each step to reach a global optimum.

Problem:

Given N intervals (s_i, e_i) , select the maximum number of non-overlapping intervals.

Idea:

Sort intervals by end time. Always pick the interval with the earliest finishing time that is compatible.

Advanced Topic 1: Ternary Search

Theory:

Ternary search finds the maximum or minimum of a unimodal function (increasing then decreasing).

Idea:

Split range $[L, R]$ into three parts using midpoints m_1 and m_2 . Compare $f(m_1)$ and $f(m_2)$ and discard one-third each step.

Advanced Topic 2: Maximum Subarray Median

Theory:

We use binary search on the answer and prefix sums.

Transform array using a guess M :

$$A[i] = \begin{cases} +1 & \text{if } A[i] \geq M \\ -1 & \text{otherwise} \end{cases}$$

Problem:

Find the maximum median of any subarray with length at least K .

Idea:

Binary search on median M . Use prefix sums to check if a valid subarray with positive transformed sum exists.